

1. Introduction and Motivation

Modern deep learning systems have achieved remarkable success, yet they rely heavily on large-scale labeled datasets. This dependence introduces a fundamental bottleneck: labels are expensive to obtain, often noisy, and rarely available at the scale of raw data. In contrast, images, videos, and multimodal signals are abundant. This mismatch between abundant data and scarce labels motivates self-supervised learning, whose goal is to learn useful representations directly from the structure of the data itself.

A second, more subtle limitation of supervised learning is its inefficient use of information. A label such as a one-hot class indicator carries very limited information, while the input itself contains rich structure: spatial patterns, compositional relationships, and semantic regularities. Self-supervised learning attempts to exploit these intrinsic regularities by designing surrogate objectives that force a model to capture meaningful structure without relying on external annotations.

2. Autoencoders and Reconstruction-Based Learning

A natural starting point is the autoencoder. Given an input x , an encoder produces a latent representation $z = f(x)$, and a decoder reconstructs the input as $\hat{x} = g(z)$. The model is trained by minimizing a reconstruction loss

$$(1) \quad \mathcal{L}_{\text{rec}} = \|x - \hat{x}\|^2.$$

The intuition is that the encoder must compress x into z while retaining sufficient information for reconstruction. However, reconstruction alone does not guarantee that z captures semantics. When the decoder is powerful, the model may focus on low-level details (e.g., textures) or learn near-identity mappings, leading to representations that are not useful for downstream tasks.

A key improvement is to make the task deliberately harder. Masked reconstruction, as in Masked Autoencoders (MAE), removes a large portion of the input and asks the model to predict the missing parts. This forces the encoder to capture global structure and long-range dependencies. The guiding principle is that harder pretext tasks can induce more informative representations.

3. Contrastive Learning

Contrastive learning takes a different perspective. Instead of reconstructing inputs, it learns by comparing samples. Two augmented views of the same input form a positive pair, while other samples act as negatives. The model is trained to bring positives closer in representation space and push negatives apart.

A widely used objective is the InfoNCE loss

$$(2) \quad \mathcal{L} = -\log \frac{\exp(q \cdot k^+ / \tau)}{\sum_j \exp(q \cdot k_j / \tau)},$$

where q is a query representation, k^+ is the corresponding positive, k_j are candidates (including negatives), and τ is a temperature parameter. In practice, embeddings are normalized as $z = h/|h|$, so that dot products become cosine similarities. This normalization is crucial: without it, the model can reduce the loss by arbitrarily increasing vector magnitudes rather than learning meaningful structure. With normalization, optimization occurs on the unit hypersphere, and learning focuses on angular relationships.

To understand the learning dynamics, it is instructive to examine the gradient. Rewriting the loss as a log-softmax,

$$(3) \quad \mathcal{L} = - \left(\frac{q \cdot k^+}{\tau} - \log \sum_j e^{q \cdot k_j / \tau} \right),$$

we obtain

$$(4) \quad \frac{\partial \mathcal{L}}{\partial q} = -\frac{1}{\tau} k^+ + \frac{1}{\tau} \sum_j p_j k_j, \quad p_j = \frac{e^{q \cdot k_j / \tau}}{\sum_i e^{q \cdot k_i / \tau}}.$$

This reveals two opposing forces: an attraction toward the positive k^+ and a repulsion from a weighted average of candidates. Importantly, the weights p_j emphasize hard negatives—those most similar to q —so the model focuses on difficult distinctions. In this sense, contrastive learning adaptively discovers which negatives matter.

4. SimCLR and MoCo

SimCLR demonstrates that a simple pipeline can be highly effective when key ingredients are chosen carefully. Strong data augmentation defines the notion of invariance; large batch sizes provide many negatives; normalization stabilizes optimization; and a projection head maps representations into a space tailored for the contrastive objective. The projection head is not a cosmetic addition: it allows the encoder to preserve general-purpose features while the projection space absorbs the specifics of the loss.

A practical limitation of SimCLR is its reliance on very large batches to supply negatives. MoCo addresses this by maintaining a queue of representations from previous iterations, effectively building a large and continually refreshed dictionary of negatives. To ensure that queued features remain compatible with current ones, MoCo employs a momentum encoder updated as

$$(5) \quad \theta_k \leftarrow m, \theta_k + (1 - m), \theta_q.$$

This slow evolution reduces inconsistency across time. Conceptually, SimCLR and MoCo solve the same problem—how to obtain many negatives—but make different trade-offs between computation and memory.

5. Non-Contrastive Learning: BYOL and SimSiam

A surprising development is that negative samples are not strictly necessary. BYOL trains a network to predict the representation of another augmented view using a teacher–student setup. The student parameters θ are optimized to match a target produced by a slowly updated teacher ξ :

$$(6) \quad \mathcal{L}_{\text{BYOL}} = |q\theta(x_1) - \text{sg}(z_\xi(x_2))|^2.$$

Here $\text{sg}(\cdot)$ denotes stop-gradient. Although the objective contains no explicit repulsion, collapse is avoided through asymmetry: the target is not updated by the loss, and a predictor network transforms the student output before matching. The teacher evolves via an exponential moving average, providing a stable reference.

SimSiam simplifies this design by removing the momentum teacher and using a symmetric objective

$$(7) \quad \mathcal{L}_{\text{SimSiam}} = \frac{1}{2} (|p_1 - \text{sg}(z_2)|^2 + |p_2 - \text{sg}(z_1)|^2).$$

At first glance, this appears to be a minor modification, but it clarifies that the essential ingredient is asymmetry (via stop-gradient and the predictor), not the momentum mechanism itself. The symmetric form mainly improves optimization efficiency. Because augmentations are

sampled randomly, one-sided training becomes effectively symmetric in expectation over many iterations; thus, the difference between symmetric and asymmetric losses is largely a matter of finite-sample optimization rather than asymptotic behavior.

6. DINO: Distribution-Based Learning

DINO replaces vector regression with distribution matching. Given logits z , a softmax produces a probability vector

$$(8) \quad p = \text{softmax}(z/T),$$

and the student is trained to match the teacher distribution using cross-entropy

$$(9) \quad \mathcal{L}_{\text{DINO}} = - \sum k p_t^{(k)} \log p_s^{(k)}.$$

These probabilities do not correspond to semantic classes; rather, they represent soft assignments over learned prototypes. The use of softmax imposes a simplex constraint, enabling distribution-level control.

Two mechanisms are critical to avoid collapse. Sharpening uses a small temperature for the teacher, producing low-entropy, confident targets. Without sharpening, the model may converge to a uniform distribution across all dimensions. Centering subtracts a running mean across samples for each feature dimension,

$$(10) \quad \tilde{z} = z - c, \quad c \leftarrow mc + (1 - m), \mathbb{E}_{\text{batch}}[z],$$

which prevents any single dimension from dominating across the dataset. Together, sharpening enforces confident predictions per sample, while centering enforces diversity across samples; their interplay stabilizes training.

DINO further introduces a multi-crop strategy. The teacher processes only global views, while the student processes both global and local views. The objective enforces consistency across these views, including cases where the student observes only partial information. This cross-scale constraint encourages representations that are robust and semantically meaningful.

DINO is particularly effective with Vision Transformers (ViTs). While ViTs provide tokenized representations and global self-attention, the key is not merely the presence of tokens but how they interact under the training objective. Self-attention allows tokens to communicate globally, and DINO’s distribution matching encourages tokens that belong to the same object to produce consistent outputs. As a result, clustering emerges at the token level, often yielding unsupervised segmentation behavior.

7. CLIP: Cross-Modal Contrastive Learning

CLIP extends contrastive learning to the multimodal setting by learning from image–text pairs. An image encoder f and a text encoder g are trained jointly so that matched pairs have high similarity while mismatched pairs are separated. The objective for image-to-text matching is

$$(11) \quad \mathcal{L}_{\text{CLIP}} = - \log \frac{\exp(\text{sim}(v_i, t_i)/\tau)}{\sum_j \exp(\text{sim}(v_i, t_j)/\tau)},$$

and a symmetric term is used for text-to-image. This is essentially the InfoNCE objective applied across modalities.

Because language provides rich and flexible descriptions, CLIP learns a shared embedding space that captures high-level semantics. A notable consequence is zero-shot learning: classification can be performed by comparing an image embedding to text embeddings of candidate labels (e.g., “a photo of a dog”), without task-specific training.

8. A Unified Perspective

Across these methods, a unifying view emerges: self-supervised learning is about designing constraints on representations. Reconstruction preserves information; contrastive learning enforces invariance via discrimination; non-contrastive methods rely on alignment with asymmetry to avoid collapse; DINO shapes distributions to balance confidence and diversity; and CLIP leverages language to inject semantics. Small changes in objectives can induce large changes in the geometry of the learned space, and effective designs arise from aligning objectives with architectural biases and data transformations.